

SPRAWOZDANIE NR 5			
Nazwa ćwiczenia	Serverless deployment		 POLITECHNIKA BYDGOSKA Wydział Telekomunikacji Informatyki i Elektrotechniki
Przedmiot	Nowoczesne Technologie Przetwarzania Danych		
Student grupa	Mikołaj Domagała gr. 2		
Data ćwiczeń	26.03.2026	13.04.2026	Data oddania sprawozdania

Cel i zakres ćwiczenia :

Celem ćwiczenia jest wdrożenie modelu ML na Google Cloud Run / Render.com; Porównanie zalet i wad wdrożeń serverless vs własny serwer.

Przebieg :

Zainstalowałem Google Cloud

```
Would you like to create one? (Y/n)? n
The Google Cloud CLI is configured and ready to use!
```

Następnie stworzyłem plik docker ignore

```
# virtual enviroment
.venv/
venv/
env/
.idea/

# pamiec podreczna
__pycache__/
*.pyc
*.pyo
*.pyd

# logi
*.log
.git/
.gitignore|
```

ntpd-dockerapi-production.up.railway.app us-west2 1 Replica

ACTIVE  **sprawozdanie** 4 minutes ago via GitHub View logs ⋮

✓ Deployment successful ▼

Public Networking

Access your application over HTTP with the following domains

 **ntpd-dockerapi-production.up.railway.app**
→ Port 8080 Copy 📄 ⚡ ✎ 🗑️

+ Custom Domain + TCP Proxy

Private Networking

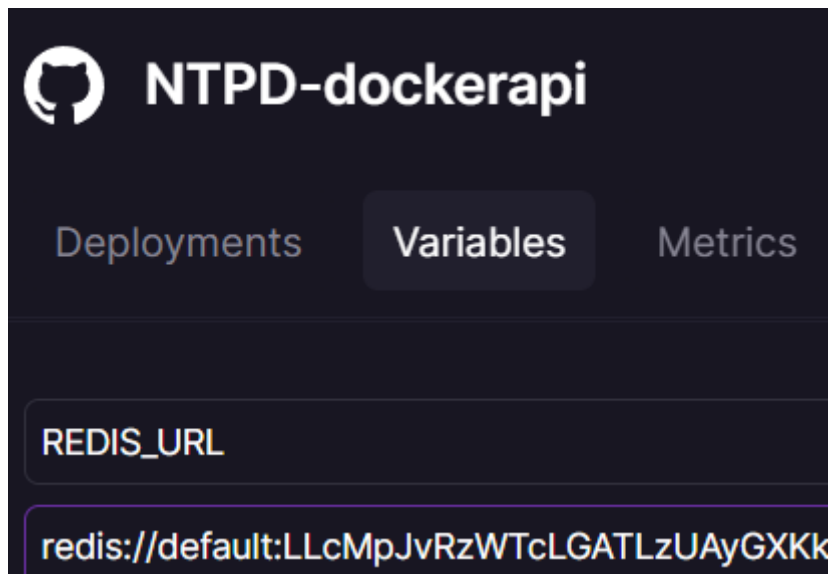
Communicate with this service from within the Railway network.

✓ **ntpd-dockerapi.railway.internal** IPv4 & IPv6
Ready to talk privately · You can also simply call me **ntpd-dockerapi**

```
< > ↻ 📖 🔗 ntpd-dockerapi-production.u... 🔗 | 🔥  
Zastosuj formatowanie stylistyczne ☐  
{ "Hello": "Test123" }
```

Connection URL show open

redis://default:*****@maglev.proxy.rlywy.net:32477 📄

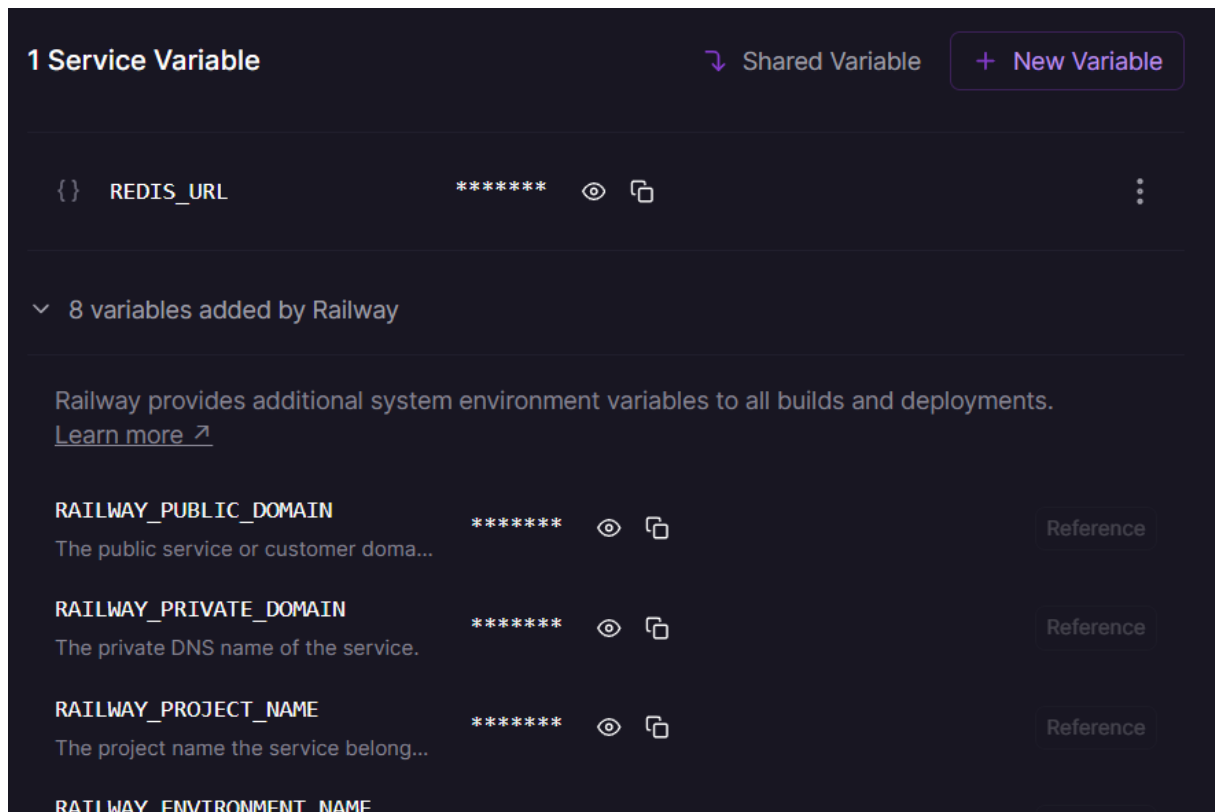


CURL:

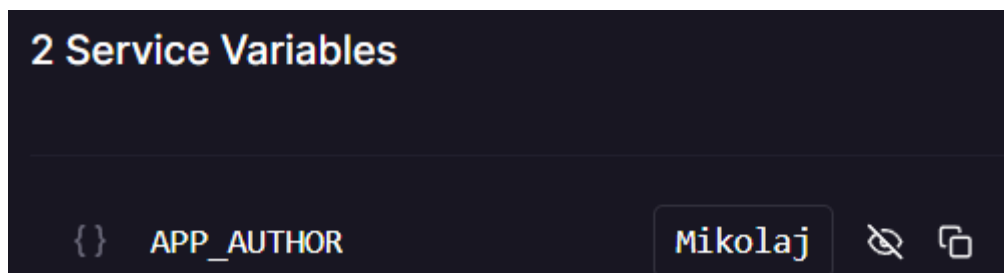
```
C:\Windows\System32>curl -X POST "https://ntpd-dockerapi-production.up.railway.app/predict" -H "Content-Type: application/json" -d "{\"hours\": 10}"  
{"input_hours":10.0,"predicted_score":102.0}  
C:\Windows\System32>
```

Wdrażając ML w sposób severless można stwierdzić że odbywa się to szybko ponieważ wystarczy połączyć z githubem i aplikacja po 2 minutach wstaje. Nie trzeba się także martwić o aktualizacje i zabezpieczenia oraz instalowanie dockera na serwerze. Wadą jest że to podejście ma ograniczenia zasobów, ponieważ ciężko tam wrzucić duże modele które wymagają specyficznych GPU.

Z kolei własny serwer np. Docker na maszynie ma te zalety pełnej kontroli, że jest dostęp do wszystkiego i można optymalizować pod konkretny model ML. Prywatność też jest tutaj kluczowa ponieważ dane nigdzie nie wychodzą. Wadą na pewno jest to że serwer chodzi 24/7 nawet jeśli nikt z niego nie korzysta. Z bezpieczeństwem też trzeba dbać samemu.



Rozszerzyłem aplikację o obsługę zmiennej środowiskowej. Wygenerowałem nową bazę danych oraz skopiowałem redis_url.



Do testowania zmiennych w konfiguracji dodałem APP_AUTHOR i Mikolaj. Następnie trochę zmodyfikowałem kod w głównej aplikacji aby odebrała tę zmienną

```
app_author = os.getenv("APP_AUTHOR", "błąd zmiennej")
@app.get("/")
def read_root():
    return {"message": f"witam cie {app_author}"}
```

No i na końcu przetestowałem endpoint

```
{"message": "witam cie Mikolaj"}
```

Jak widać zmienna została poprawnie odebrana

Wnioski :

W tym laboratorium nauczyłem się korzystać z Railway oraz obsługiwać w nim pliki DockerFile.
Nauczyłem się także obsługiwać zmienne środowiskowe bez wypisywania danych na sztywno